Chapter 3: Existing methods

In this chapter we describe a few of the statistical reconstruction techniques used for MAP estimation. Two of these, Simulated Annealing (Geman and Geman, 1984) and Iterated Conditional Modes (Besag, 1986) have proved fundamental in this area and have been fully implemented by the author; an example is used to demonstrate these methods. Other techniques have been developed which incorporate these methods and these shall be discussed later in this chapter. The intention of this chapter is to summarise these methods in a clear and concise manner, observing limitations, advantages and peculiarities while referring the reader to the works of the given authors for the detailed theoretical support.

3.1 The example

Figure 3.1 shows a 64 by 64 binary image.

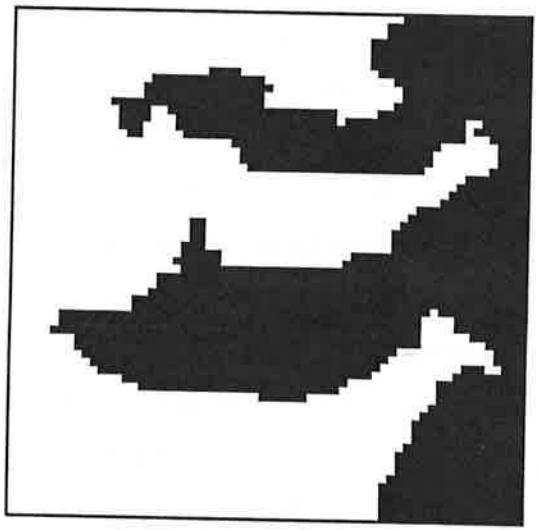

Figure 3.1

Each pixel is coloured black or white and the colour of pixel i is denoted by x_i^* which takes the value 0 for white and 1 for black. The records y_i are independently distributed as Gaussian with mean x_i^* and variance $\sigma^2=0.25$. Thus the likelihood of the record y given x is given by

$$l(y|x) = \prod_{i=1}^{n} f(y_i|x_i) = \exp\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i-x_i)^2\right]$$

The prior model which will be used in reconstruction is given by

$$p(x) \propto \exp[-\{\beta_1 Z_1(x)+\beta_2 Z_2(x)\}]$$

where $Z_1(x)$ is the number of discrepant first order pairs in the scene x, i.e., the number of pairs of first order neighbours which are of opposite colour, $Z_2(x)$ is the number of discrepant second order pairs and β_1 and β_2 are fixed positive constants. Hence the posterior probabilities may be calculated from

$$P(x|y) \propto l(y|x)p(x) \propto \exp\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i-x_i)^2\right]. \exp\left[-\{\beta_1 Z_1(x)+\beta_2 Z_2(x)\}\right]. \quad (3.1)$$

The maximisation of the posterior probability may be reformulated as the minimisation of the objective function

$$\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i-x_i)^2 + \left[\beta_1 Z_1(x)+\beta_2 Z_2(x)\right] \quad (3.2)$$

over values of x.

The first term of the objective function is a measure of the infidelity between the record and the reconstruction and the second is a measure of the roughness of the reconstruction.

At this point some comment on the choice of the β values is required. There are several different methods for establishing a "good" value but a "try it and see" approach is commonly used. If large values of β_1 and β_2 are used then the objective function is dominated by the roughness term and the reconstruction which minimises (3.2) will be smooth. If small values of β_1 and β_2 are used then the reconstruction which minimises (3.2) will correspond closely to the record. Some inference as to the correct ratio of β_1 to β_2 has been made by Silverman, Jennison and Brown (1989) for the second order model. They have shown that using $\beta_2=\beta_1/\sqrt{2}$ minimises the rotational variance of the prior model with respect to the positioning of the pixel grid on a given scene. In Chapter 6 of this thesis we propose a technique for detecting oversmoothing in image reconstruction.

Before describing any techniques we define the closest mean classifier as being the image which contributes least to the first term of the objective function i.e. the image which has most fidelity with the record. Each pixel is coloured with the colour which has mean closest to its record e.g. if $y_i=0.4$ then pixel i is coloured white since

$$\left|0.4-0.0\right| < \left|0.4-1.0\right|.$$

In iterative methods it is usual to use the closest mean classifier as an initial estimate of the true scene.

3.2 Iterated conditional modes

Besag (1986) proposed the method of Iterated Conditional Modes (ICM) which finds a local maximum of the objective function (3.2) in the following way: The closest mean classifier is used as an initial estimate of the true scene. The region is scanned and at each pixel i say, the colouring x_i is chosen to minimise (3.2) over all possible x_i with all other pixel colourings fixed. This is equivalent to choosing x_i to minimise

$$\frac{1}{2\sigma^2}(y_i - x_i)^2 + \left[\beta_1 Z_1'(x_i) + \beta_2 Z_2'(x_i)\right]$$

where $Z_1'(x_i)$ and $Z_2'(x_i)$ are the number of first and second order neighbours of pixel i which are opposite colour to x_i. Inspection of (3.1) shows that this choice of x_i maximises

$$l(y_i \mid x_i) p(x_i \mid x_{S\setminus i}),$$

where $x_{S\setminus i}$ is the colouring of all pixels except the colouring at pixel i. This is equivalent to choosing x_i to maximise

$$l(y_i \mid x_i) p(x_i \mid x_j, j \in \partial_i).$$

It follows that this choice of x_i gives the most likely colouring at each pixel based only on its record and the colouring of its neighbours.

Each scan in which every pixel is considered for updating once is known as an iteration. The order in which the region is scanned is arbitrary if updating is synchronous since only the neighbour information from the last iteration will be used. It is, however, more convenient to scan the region in a raster fashion, storing only the current colouring of each pixel. When updating a pixel, the neighbours which are to the left or below the pixel have been more recently updated than those to the right or above the pixel. Besag (1986) notes that small directional effects may be introduced if the region is scanned in this way and suggests that these effects may be lessened if the raster is changed after each iteration. In the example that follows a straightforward raster scan is used for convenience and storage reasons. When a raster scan is used, the objective function must decrease or remain constant at each updating and because there are

a finite number of possible colourings a local minimum will be achieved in a finite number of iterations.

Besag (1986) uses $\beta_1=\beta_2=1.5$ for reconstruction but our experimental results suggest that this value is too high. This conclusion has also been reached by Ripley (1986). Another approach used by Besag was to increase the value of the smoothing parameter during reconstruction, settling at a value of 1.5. This encourages fidelity with the record in the early stages of reconstruction and avoids using what may be misleading spatial information early in the process. A similar approach has been adopted by Stander (1989) who has found that stopping the process much earlier, when $\beta=0.75$, has given superior results. Using the closest mean classifier as an initial estimate of the scene corresponds to using $\beta_1=\beta_2=0$ for the first iteration.

In the example we use a constant value of β for all iterations. Figure 3.2 shows the closest mean classifier for the record. In this initial estimate of the scene approximately 16% of the pixels are misclassified. Below each of the displayed reconstructions we give the number of misclassified pixels when compared to the true scene. Ripley (1986) warns of the dangers of using misclassification rates as a measure of goodness of fit and these figures should only be regarded as a rough guide to the quality of the reconstruction. Figure 3.3 shows the reconstruction obtained using ICM with $\beta_1=1$ and $\beta_2=0$. Setting $\beta_2=0$ is equivalent to using a first order model in which pixels are considered neighbours only if they are horizontally or vertically adjacent. A more interesting reconstruction is shown in Figure 3.4 which was obtained using ICM with $\beta_1=0$ and $\beta_2=1$. Reconstructing the image in this way corresponds to reconstructing two independent images, each containing 32×32 pixels diagonally connected (like the black squares on a chequerboard). The two separate regions cannot interact and this leads to the unusual reconstruction shown in Figure 3.4. A more sensible model using $\beta_1=1$ and $\beta_2=\beta_1/\sqrt{2}$ gives Figure 3.5 which appears satisfactory. Figure 3.6 is obtained using ICM with $\beta_1=4$ and $\beta_2=\beta_1/\sqrt{2}$; using such large values discourages sharp edges or protrusions in the image and this is illustrated clearly here.

Although Besag suggests the use of the closest mean classifier as the initial estimate ICM may be applied using any colouring as the initial estimate and the effect of this can be seen in Figures 3.7 and 3.8 which were obtained using ICM

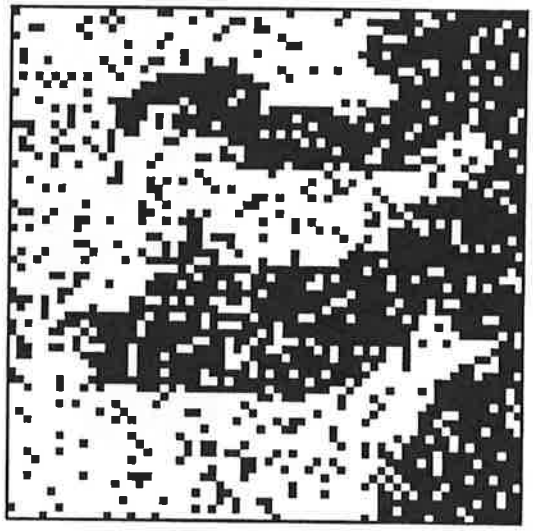

Figure 3.2
The closest mean classifier
Number of pixels misclassified : 673

Figure 3.3
Reconstruction technique: ICM
Parameters: $\beta_1 = 1.0$
$\beta_2 = 0.0$ (first order model).
Initial estimate: Closest mean classifier
Number of pixels misclassified : 93

Figure 3.4
Reconstruction technique: ICM
Parameters: $\beta_1 = 0.0$
$\beta_2 = 1.0$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 164

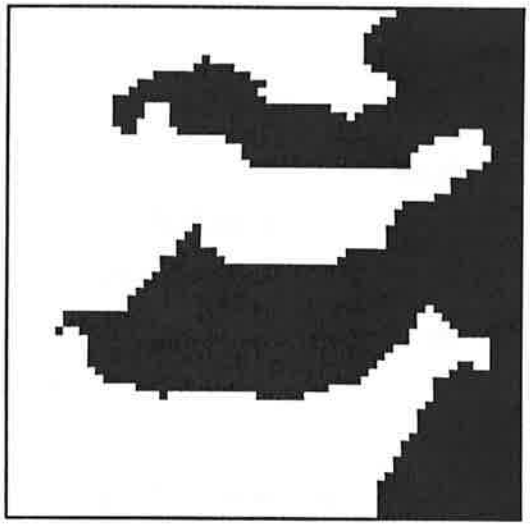

Figure 3.5
Reconstruction technique: ICM
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 62

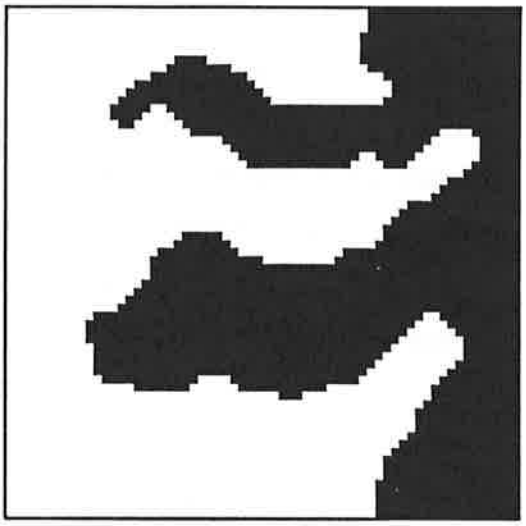

Figure 3.6
Reconstruction technique: ICM
Parameters: $\beta_1 = 4.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 142

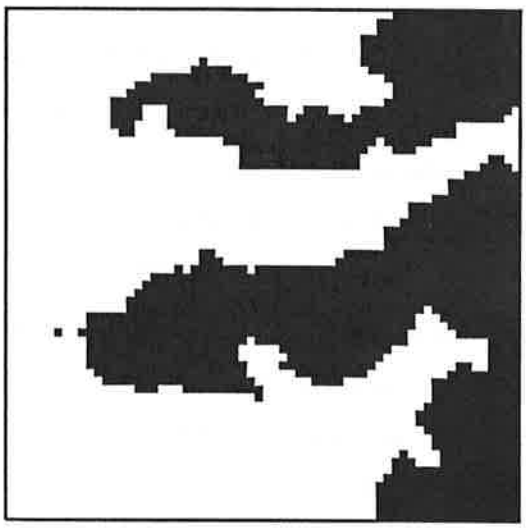

Figure 3.7
Reconstruction technique: ICM
Parameters: $\beta_1 = 0.8$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
Initial estimate: All white scene
Number of pixels misclassified : 178

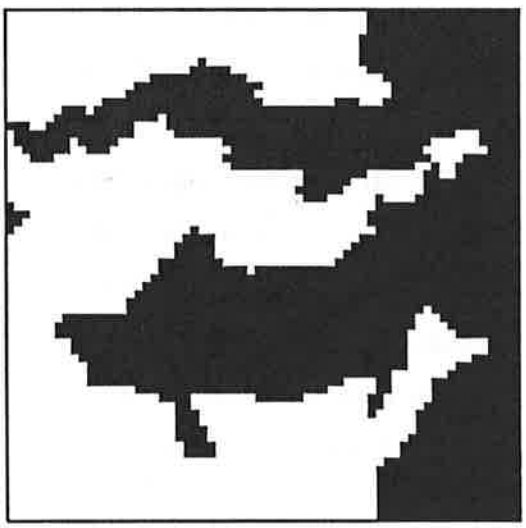

Figure 3.8
Reconstruction technique: ICM
Parameters: $\beta_1 = 0.8$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
Initial estimate: All black scene
Number of pixels misclassified : 244

with $\beta_1=0.8$, $\beta_2=\beta_1/\sqrt{2}$ and initial estimates which were all white and all black respectively. When larger values of β_1 and β_2 were used the reconstruction varied little from the initial estimates in both cases.

When processing large images we avoid unnecessary computation by storing the coordinates of pixels whose colourings have changed in the current iteration. If the number of these is small, only pixels whose neighbours have changed colour in the last iteration are considered in the next iteration. In general, only two complete iterations are required in reconstruction although Figures 3.7 and 3.8 required 6 complete iterations before the number of changes became sufficiently small.

3.3 Simulated annealing

Geman and Geman (1984) liken images to physical systems in which the states of atoms or molecules interact in a lattice-like structure. In the physical context of annealing an energy is minimised by gradually reducing the temperature in the system; a gradual cooling is essential to avoid ending in a local minimum. The objective function is viewed in a similar way and random changes are allowed in the reconstruction process which increase the objective function in the hope that a better minimum will eventually be achieved. The essence of their approach uses a "stochastic relaxation" algorithm which generates a sequence of images that converges to the MAP estimate with probability 1.

The image is scanned in what is typically a raster fashion although the pixels may be considered for updating in any order as long as all pixels are visited infinitely often. Each n visits is known as a sweep of the image. Considering one pixel at any given time, the posterior probability $p_i(l)$ of colouring l at pixel i given the current colouring of the rest of the region is calculated for each of the $l=1,\ldots,L$ different colourings. Simulated Annealing requires that colouring k is then assigned to pixel i with probability

$$\frac{\{p_i(k)\}^{\frac{1}{T}}}{\sum_{l=1}^{L}\{p_i(l)\}^{\frac{1}{T}}}$$

where T is the "temperature" of the process. The use of a temperature and cooling schedule follows directly from the physical analogy where gradual cooling isolates

low energy states which correspond to the most probable states in the posterior distribution. Theoretically the algorithm must run for an infinite amount of time with a logarithmic decrease in T for convergence to the global maximum to be guaranteed (see Gidas, 1985a). In practice a sensible cooling schedule must be used and truncated at some point in order that a reconstruction may be obtained. Geman and Geman advocate the use of the schedule

$$T(k) = \frac{C}{\log(1+k)} \qquad 1 \leq k \leq K$$

where K is the total number of sweeps and C is a fixed constant, usually taken to be between 2 and 5. Geman and Geman recommend the use of C=3 or 4.

The example is used to illustrate the effect of these parameters and the use of the closest mean classifier as an initial estimate of the true scene. Figure 3.9 shows the reconstruction obtained using the objective function (3.1) with β_1=1.0, $\beta_2=\beta_1/\sqrt{2}$, C=3.5, K=1000 and the closest mean classifier as an initial estimate of the true scene. These are typical values which we have found to give good results in most cases.

We first examine the effect of the C parameter. If the temperature is large then changes in pixel colourings depend less on the posterior probabilities and are more random. If the temperature is small then the changes are biased towards colourings which reduce the value of the objective function. When C is large the temperature is very high at the start of the process, increasing the likelihood that groups of pixels will change colour, leading to a different minimum of the objective function. If in the final sweep, the temperature remains comparatively high, changes may occur which significantly increase the value of the objective function, producing an image which has lower probability. This is illustrated in Figure 3.10 where C=7.0; the isolated pixel colourings occur as a result of changes made in the final sweep.

The value of K, the total number of sweeps, is also of interest. The theory which is used to justify the use of annealing suggests that the process should be allowed to run for as long as possible to maximise the chances of reaching the MAP estimate. In practical applications, where the size of the region and the number of different colourings of each pixel increase the computational burden, truncation must occur at some point. Figures 3.11 and 3.12 show the images obtained using K=33 and 10000 respectively. (We use the value K=33 in order

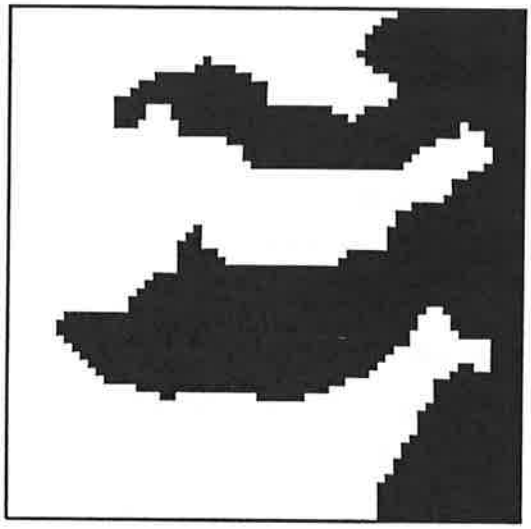

Figure 3.9
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 1000$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 57

Figure 3.10
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 7.0$
$K = 1000$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 88

Figure 3.11
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 33$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 68

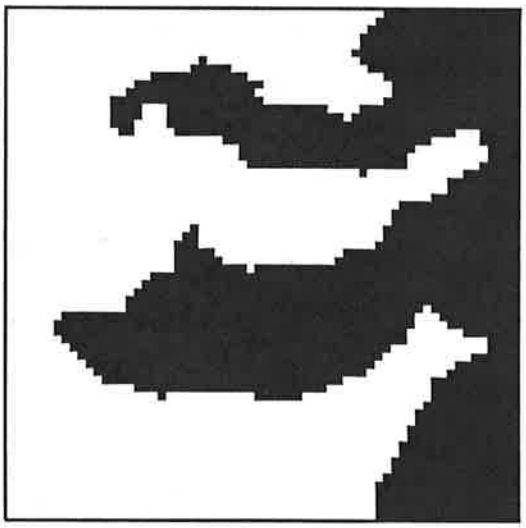

Figure 3.12
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 10000$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 56

that the temperatures at the final sweep for this reconstruction and the reconstruction shown in Figure 3.10 were approximately equal.) In general we have found that better reconstructions are obtained when K is chosen as large as possible.

We have already demonstrated the dependence of ICM on the initial estimate. Figures 3.13 and 3.14 show the reconstructions obtained by simulated annealing using all white and all black initial estimates respectively. Although these scenes differ slightly the initial estimate does not appear to have had a detrimental effect in either case. Clearly, if the process was terminated earlier any effect would be more obvious.

Isolated pixel colourings like those shown in Figures 3.10 and 3.11 may be avoided if, after the K sweeps, an additional sweep is executed with $T=0$. This is equivalent to one iteration of Iterated Conditional Modes and can only reduce the value of the objective function. Figures 3.15 and 3.16 show the reconstructions when this extra sweep has been applied to Figures 3.9 and 3.11 respectively.

3.4 Exact MAP estimation for binary images

Grieg, Porteous and Seheult (1989) show that for a two colour scene the MAP estimate may be found exactly. The problem is reformulated as a minimum cut problem in a capacitated network and the Ford-Fulkerson labelling algorithm is used to find the solution.

The likelihood function $l(y|x)$ is rewritten as

$$\prod_{i=1}^{n} f(y_i|x_i) = \prod_{i=1}^{n} f(y_i|x_i=1)^{x_i} f(y_i|x_i=0)^{1-x_i}$$

and the prior model as

$$p(x) \propto \exp\left[\frac{1}{2}\sum_{i=1}^{n}\sum_{j\in\partial_i}\beta_{ij}\mathrm{I}(x_i=x_j)\right]$$

where $\beta_{ij}=\beta_{ji}$ and $\mathrm{I}(x_i=x_j)=1$ if $x_i=x_j$ and 0 otherwise. Ignoring an additive constant we may write

$$\ln P(x|y) = L(x|y) = \sum_{i=1}^{n} x_i\lambda_i + \frac{1}{2}\sum_{i=1}^{n}\sum_{j\in\partial_i}\beta_{ij}\mathrm{I}(x_i=x_j)$$

where $\lambda_i = \ln\{f(y_i|x_i=1)/f(y_i|x_i=0)\}$, the log-likelihood ratio at pixel i.

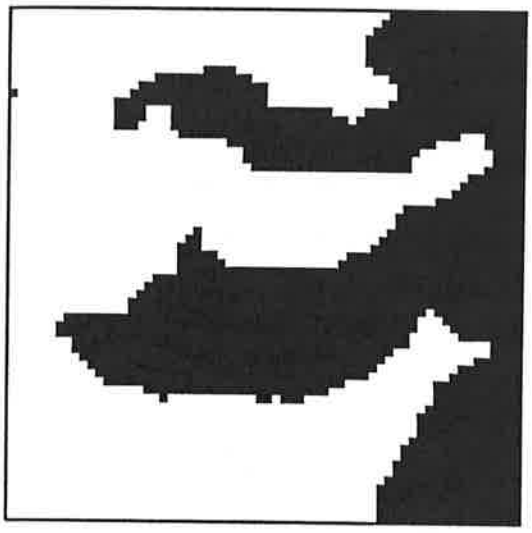

Figure 3.13
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 1000$
Initial estimate: All white scene
Number of pixels misclassified : 61

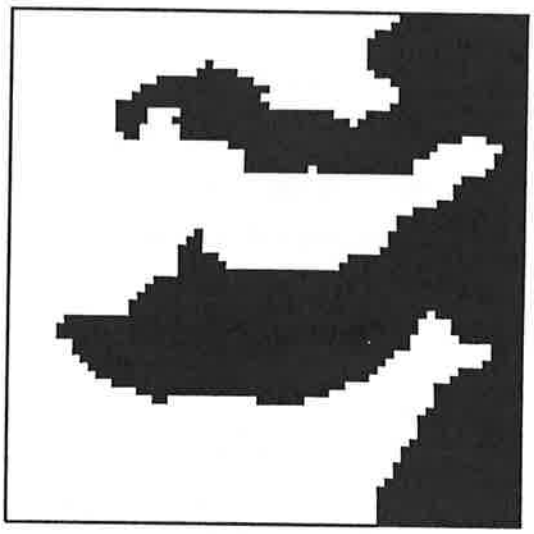

Figure 3.14
Reconstruction technique: Simulated Annealing
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 1000$
Initial estimate: All black scene
Number of pixels misclassified : 51

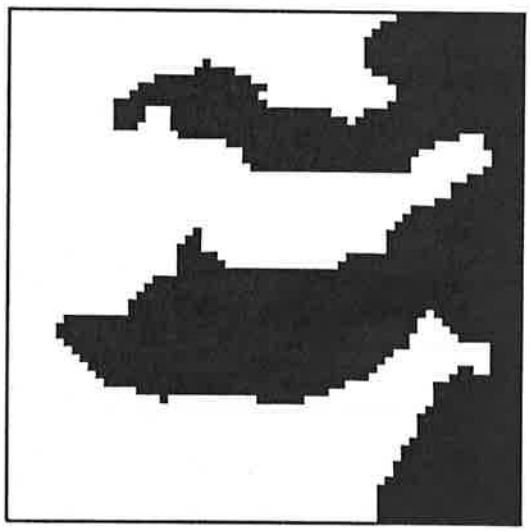

Figure 3.15
Reconstruction technique: Simulated Annealing including an additional sweep with $T=0$
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 1000$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 51

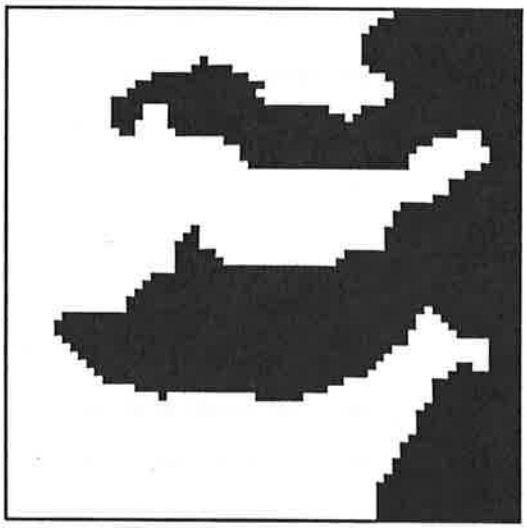

Figure 3.16
Reconstruction technique: Simulated Annealing including an additional sweep with $T=0$
Parameters: $\beta_1 = 1.0$
$\beta_2 = \frac{\beta_1}{\sqrt{2}}$
$C = 3.5$
$K = 33$
Initial estimate: Closest mean classifier
Number of pixels misclassified : 51

Consider a capacitated network containing a source, a sink and n interior vertices called nodes. There are a number of arcs from the source to interior nodes which allow flow in that direction. A number of interior nodes are connected to the sink by arcs which allow flow in that direction. In addition to these arcs, flow is permitted along arcs between interior nodes. Each of the arcs in the network has a capacity. This is the maximum flow in either direction along the arc. An example of a network is shown in Figure 3.17 although the capacities of the arcs are not shown.

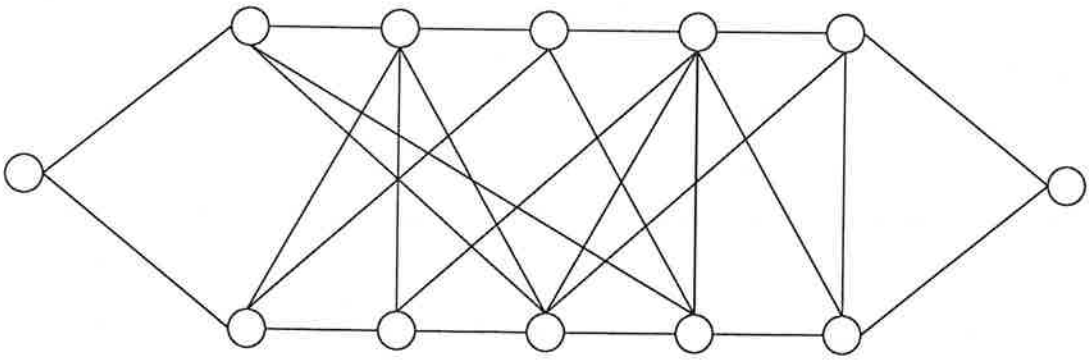

Figure 3.17

We define a *cut* of the network to be a partition of all the nodes into two sets. The first of these sets, S_1, contains the source and the other, S_2, contains the sink. Every interior node is contained in just one of the sets. The value of the cut is given by

$$C(S_1,S_2)=\sum_{k\in S_1}\sum_{l\in S_2} c_{kl}$$

where c_{kl} is the capacity of the arc between node k and node l.

The maximum flow problem is to maximise the flow from the source to the sink, using the arcs in the network. Ford and Fulkerson (1962) show that the maximum flow is equal to the minimum cut and provide an algorithm for finding the maximum flow.

In the context of imaging, the problem may be reformulated in the following way. The n pixels are regarded as interior nodes. Node i is connected to the source by an arc with capacity λ_i if $\lambda_i>0$, otherwise it is connected to the sink by an arc with capacity $-\lambda_i$. Node i is connected to node j by an arc with capacity β_{ij}. We refer to the source and sink as nodes s and t respectively. For any binary image x we may form the sets $B=\{s\}\cup\{i;x_i=1\}$ and $W=\{t\}\cup\{i;x_i=0\}$. These

two sets define a partition of the network which is a cut. The value of the cut is

$$C(x)=C(B,W)=\sum_{k\in B}\sum_{l\in W}c_{kl}$$

which may be written as

$$\sum_{i=1}^{n}x_i\ \max(0,-\lambda_i)+\sum_{i=1}^{n}(1-x_i)\ \max(0,\lambda_i)+\sum_{i=1}^{n}\sum_{j\in\partial_i}\beta_{ij}\mathrm{I}(x_i\neq x_j)$$

which differs from $-L(x|y)$ by a term which does not depend on x. The image x which maximises $L(x|y)$ corresponds exactly to the partition which gives the minimum cut, the set S_1 containing nodes corresponding to black pixels and S_2 containing nodes corresponding to white pixels. Hence, the MAP image estimate may be found by maximising the flow in the related network problem.

The Ford Fulkerson labelling algorithm for maximising the flow through a network

We use f_{ij} to represent the flow from node i to node j and denote the *excess capacity* of this arc by $d_{ij}=c_{ij}-f_{ij}$.

Step 0. Set $f_{ij}=0\ \forall ij$. Define the index of the source to be 0.

Label the source with the ordered pair $(-1,\infty)$.

Step 1. We form the set N_1 of nodes which are connected to the source by arcs with positive excess capacities. We use the index m for a general node in N_1. Label each of the nodes in N_1 with the ordered pair (e_m,p_m), where

$$e_m=d_{0m}$$
$$p_m=0$$

Step 2. We form the set N_2 of all unlabelled nodes which are connected to nodes in N_1 by arcs with positive capacities in the following way: Choose the node in N_1 with the smallest index; say it is node k. Add to N_2 the **unlabelled** nodes which are joined to node k by arcs with positive excess capacities. We use the index m for a general node in N_2. Label node m in N_2 with the ordered pair (e_m,p_m), where

$$e_m=\min\{d_{km},e_k\}$$
$$p_m=k$$

Observe that e_m is the minimum of the excess capacities of the arcs from the source to node k and from node k to node m. Also, p_m denotes the node which led to node m. Repeat this for all nodes in N_1.

Step 3. Repeat Step 2 with N_r replacing N_2 and N_{r-1} replacing N_1, for r=3,4,... After a finite number of steps, we arrive at one of two possibilities:
(i) The sink has not been labelled and no other nodes can be labelled.
(ii) The sink has been labelled.

Step 4. If we are in case (i), then the current flow can be shown to be optimal and we stop.

Step 5. If we are in case (ii), we may now increase the flow. If the sink has the label (e_r, p_r) then the flow may be increased by e_r; the second label p_r indicates the node which led to the sink. Thus the route may be retraced back to the source and the flow increased by e_r in every arc along the route from source to sink.

Step 6. Remove all of the labels from the nodes and return to Step 1.

3.4.1 Improvements in the algorithm when used for MAP estimation.

The Ford Fulkerson labelling algorithm may be applied to general networks. In the context of imaging, the networks that are derived have a particular structure and it is this that may be exploited to reduce the amount of CPU time required to find the minimum cut. Figure 3.18 shows another network which may be compared with the network in Figure 3.17. The differences in the structure of these networks is clear. The network in Figure 3.18 is of the type that will be encountered in imaging problems. All internal nodes are connected to either the source or the sink and typically approximately half will be connected to each.

The main disadvantage of the algorithm is that once a path from the source to the sink has been found and the capacities updated, other incomplete paths will be discarded as the process begins again. Figure 3.19 shows the paths that have been found up to Step 3 of the algorithm, unused arcs are shown as dotted lines. The information supplied by all but one of these will be discarded. When the number of pixels in the region is large Step 1 of the algorithm must be repeated many times and the bulk of the information which is found is not used. Figure

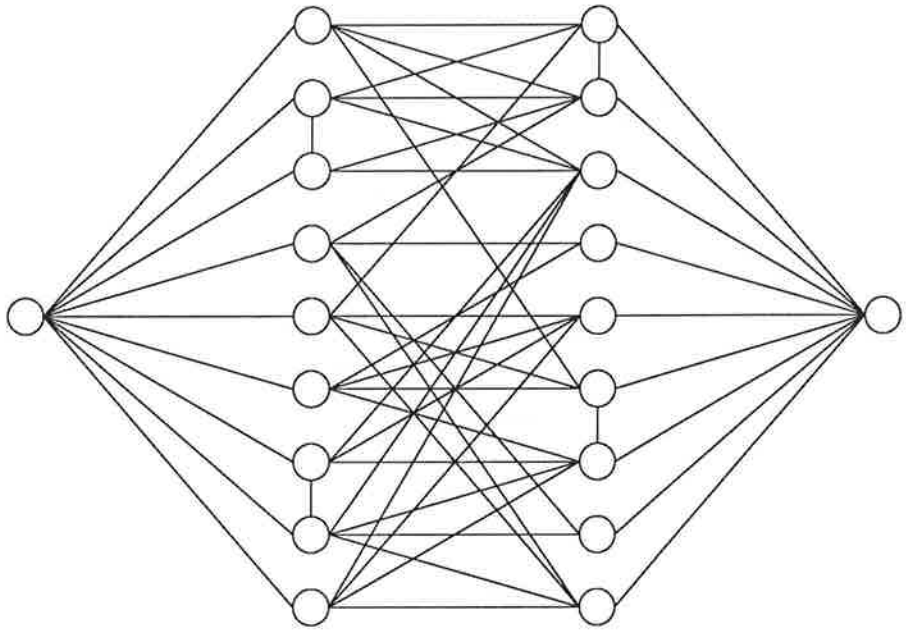

Figure 3.18

3.19 illustrates that there were several paths which would have reached the sink had the process been continued. Notice also that most of these paths would have been unaffected by updating the capacities of the successful path.

Grieg, Porteous and Seheult achieved a twelve fold reduction in CPU time by using the algorithm to find the MAP estimate for sub-regions of the image; they combine these sub-regions and then find the MAP estimate for the whole scene. In terms of the network, sections are considered in isolation and the flow maximised in that section. (Each section includes the source and the sink.) When the flow in each section is maximal all previously restricted arcs are opened and the flow increased in the original network. Figure 3.20 shows a way in which the network in Figure 3.18 may be divided up. Note that some of the arcs between internal nodes have been omitted. It is easy to see why this modification reduces the CPU requirement: if the region is small then Step 1 of the algorithm is shortened significantly.

We now propose a modified version of the labelling algorithm which exploits the special structure of the networks found in the image analysis context.

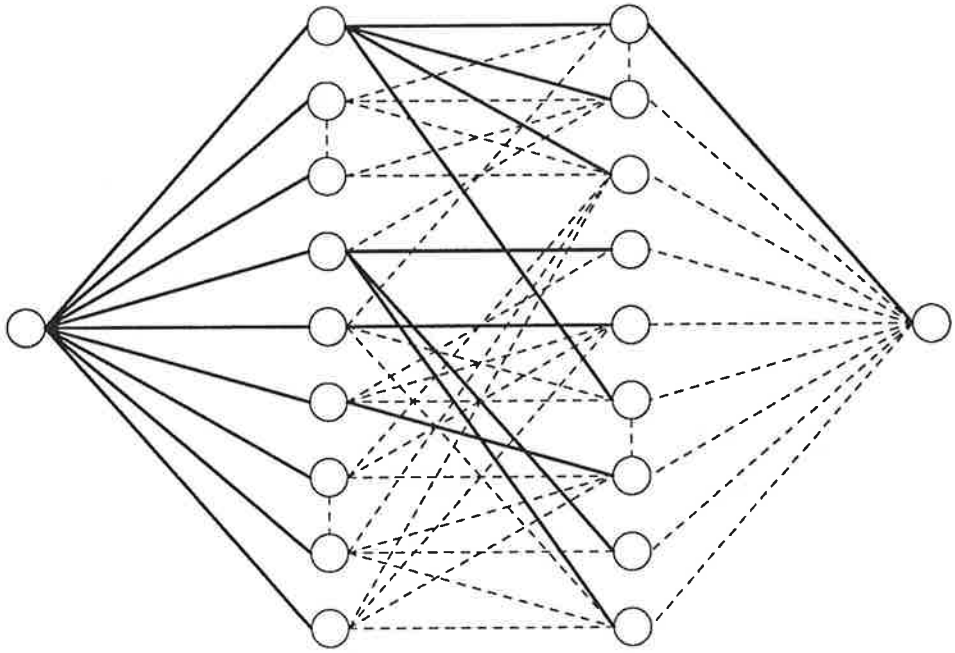

Figure 3.19

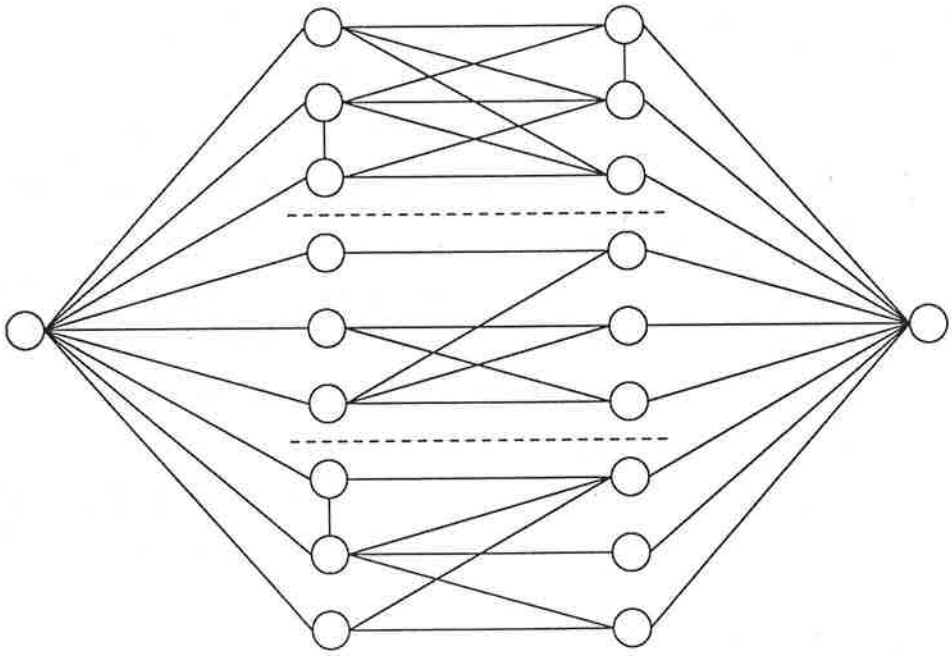

Figure 3.20

The Modified Algorithm

Step 0 Set $f_{ij}=0\ \forall ij$. Define the index of the source to be 0.

Label the source with the ordered pair $(-1,\infty)$.

Step 1 We form the set N_1 of nodes which are connected to the source by arcs with positive excess capacities. Label each of the nodes in N_1 with 0.

Step 2 We form the set of all unlabelled nodes which are connected to nodes in N_1 by arcs with positive capacities the following way:

Choose the node in N_1 with the smallest index; say it is node k. Add to N_2 the unlabelled nodes which are joined to node k by arcs with positive excess capacities. If N_2 contains the sink then remove the sink from N_2 and add k to the set T of nodes which are the penultimate nodes on successful paths from the source to the sink. Consider all nodes in N_1 in this way. Label each unlabelled node in N_2 with k, the node which led to that node.

Step 3 Repeat Step 2 with N_r replacing N_2 and N_{r-1} replacing N_1, for $r=3,4,\ldots$ After a finite number of steps no other nodes can be labelled.

Step 4 If the set T contains no nodes then the current flow can be shown to be optimal and we stop. Otherwise we increase the flow in the following way:

For the first node in T we retrace the path that led to this node and and find the minimum of the excess capacities of the arcs on this path and the capacity of the arc from this node to the sink. The excess capacities on this path may now be updated. Consider now the second node in T, we again retrace the path and find the minimum excess capacity of the path from the source to the sink for which this is the penultimate node. Note that this minimum excess capacity may have been reduced in updating the path to the first node in T. If the minimum capacity is greater than zero then we update the excess capacities on this path. All the remaining nodes in T are also considered in this way.

Step 5 Return to Step 1.

The reduction in CPU time that will be achieved using this algorithm will depend chiefly on the number of paths that are found from the source to the sink whose minimum flow is unaffected by increasing the flow in other paths.

Suppose that two complete paths share a common arc and that this arc has the minimum capacity for one or both of the paths. When the flow is updated in the first path the minimum capacity of the second path has been reduced and thus the flow may not be increased by as much. We use the network in Figure 3.18 to illustrate this difficulty. Figure 3.21 shows the paths that have been found to Step 3 of the algorithm, unused paths are shown as dotted lines. If the capacities of all the arcs are equal then of the nine paths that we have found we can increase the flow in only 4 of these. We now propose a further modification to Step 2 of the algorithm.

Step 2 Let N_2 denote the set of all unlabelled nodes which are joined to nodes in N_1 by arcs with positive excess capacities. N_2 is formed in the following way:

Choose the node in N_1 with the smallest index; say it is node k. Add to N_2 *only one* unlabelled node which is joined to node k by an arc with positive excess capacity. If N_2 contains the sink then remove it from N_2 and add k to the set T of nodes which are the penultimate nodes on successful paths from the source to the sink. Consider all nodes in N_1 in this way. When all the nodes in N_1 have been considered for one connection to an unlabelled node we return to the beginning of the set and consider all nodes for a further connection. We repeat this until no more connections are possible. We use the index m for each node in N_2. Label each unlabelled node in N_2 with k, the node which led to node m.
Observe that this modification will give identical sets $N_1,\ldots,N_r$ to those found using the previous algorithm. It will, however, lead to different choices of paths between these nodes. Figure 3.22 shows the paths that may be found if this new method is adopted. In the example that we have shown, all of the complete paths are distinct and this increases the efficiency of the algorithm.

Results

Figure 3.23 shows the MAP estimate for the example discussed earlier in this chapter. It was obtained using $\beta_1=1.0$ and $\beta_2=\beta_1/\sqrt{2}$ and has only 49 misclassified pixels. The computer program that has been implemented is capable of using the raw Ford-Fulkerson algorithm, the partitioned version suggested by Grieg *et al.* and the modified algorithm both with and without partitioning. The estimate shown in Figure 3.23 can be obtained in each of these 4 ways and Table

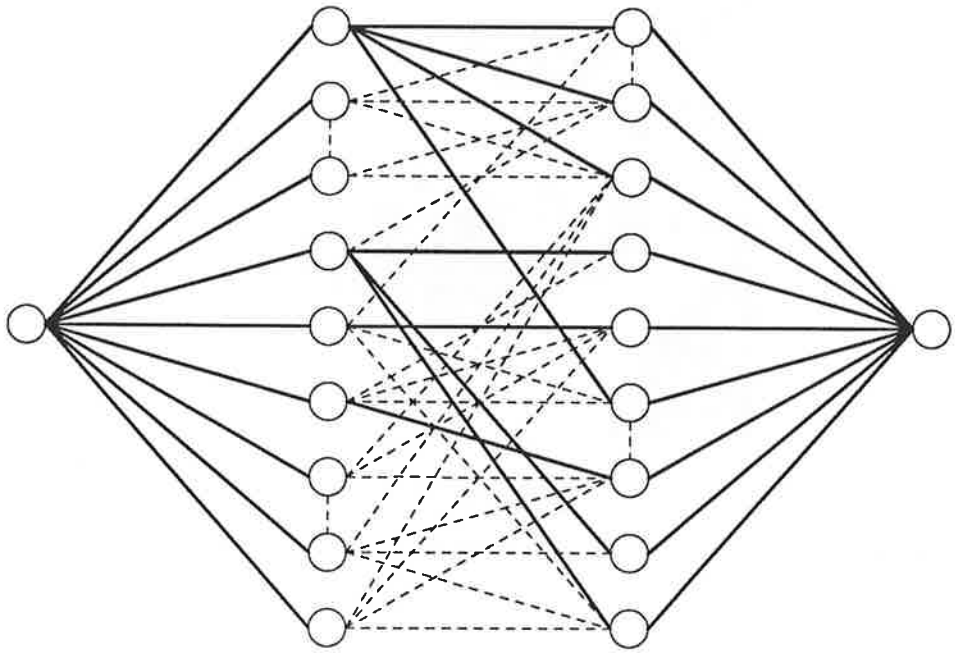

Figure 3.21

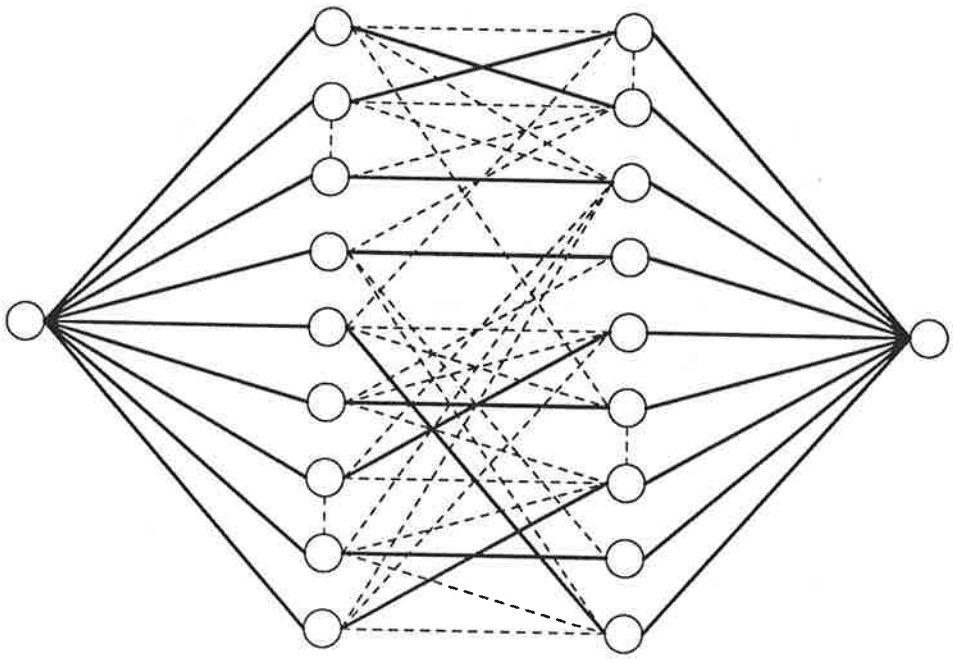

Figure 3.22

Figure 3.23

3.1 below shows the CPU requirement on a Sun 4 workstation for each case.

		using the partitioned version: Yes	using the partitioned version: No
using the modified algorithm	Yes	19.6	15.1
	No	73.0	770.0

Table 3.1

Seconds of CPU time required for a 64 by 64 binary image where $\sigma^2=0.25$, $\beta_1=1.0$ and $\beta_2=\beta_1/\sqrt{2}$.

In the partitioned version the flow is first maximised in separate 4 by 4 regions followed by 16 by 16 regions and then on the whole region. For the general m by n region we use approximately $m/16$ by $n/16$ regions followed by $m/4$ by $n/4$ regions and then the whole region, although if the sub regions are very small then we progress directly to the next level. Grieg *et al.* point out that any sensible choice will lead to a substantial reduction in CPU time. A larger example was considered to illustrate the massive reduction that may be obtained using the

modified algorithm. A 128 by 128 binary scene was perturbed by additive Gaussian noise with zero mean and variance 0.9105. This level of the variance gives a 30% misclassification rate in the closest mean classifier and was used in the examples shown in Grieg *et al.* (1989). The results are obtained using a second order neighbourhood with equal weightings for all of the neighbours and with β=0.3.

		using the partitioned version: Yes	using the partitioned version: No
using the modified algorithm	Yes	115	111
	No	630	15921

Table .2
Seconds of CPU time required for a 128 by 128 binary image
where σ^2=0.9105 and $\beta_1=\beta_2$=0.3.

In both of the examples shown above the fastest results were obtained using the modified algorithm alone. Combining the partitioned version with the modified algorithm has little effect on the CPU requirement. The reduction obtained by Grieg *et al.* using the partitioned version can also be seen here. Both the variance and the value of β that is used affect the time required for processing. Where the variance is small the arcs from the source and sink to the internal nodes have smaller capacities and hence the flow may be maximised quicker. When β is small the capacities of the arcs between internal nodes are small and again the flow may be maximised quicker. For a 128 by 128 example with variance 0.1 and β=0.3 the modified algorithm takes only 49 seconds.

3.4.2 Comparing the approximate MAP reconstructions with the exact MAP estimate.

In Sections 3.2 and 3.3 we have compared the reconstructions that have been obtained with the original true scene. In addition we may now compare these reconstructions with the exact MAP estimate shown in Figure 3.23.

Figure	Number of different pixels
3.5	23
3.9	18
3.12	13
3.13	16
3.14	18
3.15	8
3.16	8

Table 3.3
Comparing earlier reconstructions with exact MAP estimate

These numbers allow us to assess directly the capability of different methods to find the MAP estimate, independently of whether the MAP estimate is itself a good estimate of the true image.

3.5 Other methods

The methods that have been described are not the only techniques for MAP estimation, although many recently developed techniques do incorporate ICM or Simulated Annealing. We now describe two MAP estimation techniques which use alternative approaches to the maximisation.

Derin *et al.* (1984) consider horizontal strips of the image and uses a dynamic programming algorithm to find the MAP estimate for each strip. These overlapping strips are combined to give a near optimal MAP estimate of the true scene. This technique depends heavily on the correlation between the colourings of pixels dropping rapidly as the vertical distance between them increases. If the strips are narrow then it is more likely that vertical correlations will exist and since the method can only feasibly be operated with a strip width of 2 to 4 pixels this may be a drawback. In subsequent work Derin *et al.* (1985) show that the

computational burden usually incurred when processing images containing several different region types may be avoided to some extent if the region types may be ordered. Consider a region which contains 4 region types with means r_1, r_2, r_3 and r_4; with $r_1<r_2<r_3<r_4$. Derin et al. first assume that the image contains only 2 region types and that these have means r_2 and r_3. The dynamic programming algorithm is applied and a reconstruction is obtained. The pixels which have been coloured with r_2 are then considered in isolation and the algorithm applied assigning colours r_1 and r_2. The region which was coloured with mean r_3 is then reconstructed using the colours r_3 and r_4. This gives a 4 colour reconstruction for the whole scene. Good results are obtained using this method. The examples that are shown in Derin *et al.* (1985) consider up to 8 levels but clearly the method may be extended to handle any number of levels. Combining this technique with exact MAP estimation (Greig *et al.*) as described in Section 3.4 we may obtain the exact MAP estimate at each two colour stage as compared with the near optimal estimate offered by Derin *et al.* (1985). Although the exact MAP estimate may be obtained at each stage, combining these estimates in this way will not give the global MAP estimate for the k level case. An method which uses this approach is described in Chapter 6.

Gidas (1989) proposed a "coarse-to-fine" method which reconstructs the image, using simulated annealing, on a coarser grid than that on which the record was collected. This reconstruction is then used in turn to supply information to the process at a finer level. This process is repeated until a reconstruction has been obtained at the scale at which the record was originally collected. Gidas states

> *"The intuitive basis of the method .... lies in the fact that in image processing problems, ... as well as in many other massive computational tasks we are facing today, one deals with cooperative features that occur over many spatial (or temporal) scales with various interscale interactions."*

and continues

> *"Furthermore, it is intuitively clear that multilevel multiresolution processing can reveal useful information for representation, interpretations, or recognition tasks."*

The method uses a renormalisation group algorithm which provides consistency between the model at different grid scales. We shall not describe the algorithm in more

detail here but in Chapter 4 of this thesis we propose a method which uses a similar "cascade" approach which incorporates ICM at each stage.

3.6 Parameter estimation

In the context of imaging there are in general two parameter vectors that may be unknown. The first of these is the noise parameter; where the distribution of the noise is known, say it is Gaussian, the mean and variance of the distribution must be estimated. For this case the mean is usually zero and where the variance is unknown it can often be accurately estimated from the study of training data.

In addition to any noise parameters that must be estimated we require a suitable value of β, the "smoothing" parameter which affects the smoothness of the reconstruction.

Coding methods

Cross and Jain (1983) generate Markov Random Fields using the Metropolis algorithm and have notable success in simulating binary and grey scale textures using parameters which have been estimated from observed textures. They use coding methods, first introduced by Besag (1972), to estimate the parameters. Given a colouring of the region, the pixels are divided into sets which do not contain neighbours, this division does not depend on the colourings of the pixels. For each set, estimates are obtained which maximise the product of the conditional likelihoods of the colourings of the pixels given the colourings of their neighbours. The advantage of this method is that under the assumptions of the model, the colourings of each of the pixels in any set are conditionally independent of one another given the colourings of all other pixels, and thus maximum likelihood estimates can be obtained from the conditional likelihoods. For a first order model the pixels are coded 1 or 2 such that pixels which are horizontally or vertically adjacent are coded differently, giving a chequerboard effect. Each coded region is then treated separately giving two sets of observations, each providing one estimate of the parameter. The estimates from the two sets may then be combined in an appropriate way to give one estimate. Higher order models require different codings and the estimates for different codings are obtained in the same way.

Besag (1986) suggests that a neater and more efficient procedure is to use maximum pseudo likelihood, which finds the estimate which maximises the product of the conditional likelihoods over all the pixels in the region. When maximum pseudo likelihood is used, Besag suggests that boundary pixels are excluded from the likelihood because of the artificiality of the model there. A more detailed overview of maximum pseudo likelihood is given by Geman and Graffigne (1986).

Pseudo-likelihood and maximum likelihood methods may be easily used in iterative reconstruction techniques. Such techniques are sometimes known as adaptive methods and are used in conjunction with simulated annealing by Lakshmanan and Derin (1989) and Geman (1985). Besag (1986) outlines a procedure for estimating both the noise parameters and a suitable smoothing parameter during ICM. Frigessi and Piccioni (1988) consider the case of the binary channel, where each pixel changes colour with unknown probability ε, independently of the others. They propose a method for finding estimates of both ε and β which they show are consistent if the region is regarded as having a free boundary. The consistency of maximum likelihood and pseudo likelihood estimates for MRF parameters is also examined by Gidas (1986).

As we have mentioned earlier a "try it and see" approach is commonly used and, where the method is computationally inexpensive, it may be desirable to produce several reconstructions using different β parameters.

It is important to understand the influence of the method which is being used to provide the reconstruction when considering values of β. Often very different reconstructions can be obtained from different methods using the same parameters. With a local technique like ICM, the reconstruction process is such that pixels which are far apart can rarely influence one anothers colourings. Where exact MAP estimation is used long range dependencies in the model can have a large affect on the resulting reconstruction.

In addition to the works that have already been mentioned the following references are considered useful texts in this area. For general overviews of the statistical problem see Grenander (1983), Ripley (1986) and Ripley (1988a). On the use of MRF models as priors see Besag (1974), Kashyap and Chellapa (1983) and Ripley (1988b). For an introduction to parameter estimation see Grenander (1985), Grenander and Osborn (1983) and Gidas (1985b). See also Hall and

Titterington (1986) on more general parameter estimation.